

Módulo 1

M1.1 Histórico da programação orientada a objetos

Os engenheiros civis constroem vários tipos de modelos. Com maior frequência, encontram-se modelos estruturais que ajudam as pessoas a visualizar e a especificar partes do sistema e os relacionamentos existentes entre essas partes. Dependendo de ser mais importante o interesse comercial ou a questão da engenharia, os engenheiros também poderão elaborar modelos dinâmicos – por exemplo, com a finalidade de ajudá-los a estudar o comportamento de determinada estrutura em relação a tremores de terra. Cada tipo de modelo é organizado de modo diferente e cada um tem o seu próprio foco. No caso do software, existem várias maneiras de se definir um modelo. As duas maneiras mais comuns são provenientes da perspectiva de um algoritmo ou da perspectiva orientada a objetos.

A visão tradicional no desenvolvimento de softwares adota a perspectiva de um algoritmo. Nessa visão, o principal bloco de construção do software é o procedimento ou função. Essa perspectiva conduz os desenvolvedores a voltar seu foco de atenção para questões referentes ao controle e à decomposição de algoritmos maiores em outros menores. Não existe nenhuma grande desvantagem nessa solução, com exceção da tendência a permitir sistemas instáveis. À medida que os requisitos se modificam(e isso certamente ocorrerá) e o sistema cresce (o que também ocorrerá), será difícil fazer a manutenção de sistemas construídos a partir do foco em algoritmos.

A programação orientada a objetos aproveitou as melhores idéias da programação estruturada e combinou-as com novos conceitos poderosos que levam a ver na tarefa de programação uma nova luz, permitindo que um problema seja mais facilmente decomposto em subgrupos relacionados. Então, usando-se a linguagem, pode-se traduzir esses subgrupos em objetos. A visão contemporânea no desenvolvimento de software adota uma perspectiva orientada a objetos. Nessa visão, o principal bloco de construção de todos os sistemas de software é o objeto ou classe. Explicando de maneira simples, um objeto é alguma coisa geralmente estruturada a partir do vocabulário do espaço do problema ou do espaço da solução; uma classe é a descrição de um conjunto de objetos comuns. Todos os objetos tem uma identidade (você pode atribuir-lhes nomes ou diferenciá-los dos demais objetos de alguma maneira), um estado (costuma haver dados associados) e um comportamento (você poderá fazer algo com o objeto ou ele poderá fazer algo com outros objetos).

Por exemplo, considere uma arquitetura simples com três componentes para um sistema de cobrança, incluindo a interface para o usuário, uma camada intermediária e o banco de dados. Na interface para o usuário, você encontrará objetos concretos como botões, menus e caixas de diálogo. No banco de dados haverá objetos concretos como tabelas, que representam entidades provenientes do domínio do problema, incluindo clientes, produtos e pedidos. Na camada intermediária, você encontrará objetos como transações e regras de negócios, além de visões de alto nível relacionadas às entidades do problemas, como por exemplo, clientes produtos e pedidos.

O método orientada a objetos para desenvolvimento de software é, com certeza, uma parte do fluxo principal, simplesmente porque tem sido provado seu valor

para a construção de sistemas em todos os tipos de domínios de problemas, abrangendo todos os graus de tamanho e complexidade. Além disso, muitas linguagens, sistemas operacionais e ferramentas contemporâneos são, de alguma forma, orientados a objetos, fortalecendo a visão de mundo em termos de objetos. O desenvolvimento orientado a objetos fornece os fundamentos conceituais para a montagem de sistemas a partir de componentes com a utilização de tecnologias como J2EE e .Net.

Ao contrário do que muita gente pensa, a programação orientada a objetos (POO) não é uma inovação tão recente. A POO teve origem na década de 1960, na Noruega, no Centro Norueguês de Computação, pelos pesquisadores Kristen Nygaard e Ole-Johan Dahl, vistos na figura a seguir (REGNESTRAL, 2014). [L1] Nessa época, foram criados os conceitos sobre classe e herança (que serão apresentados em detalhes) implementados na linguagem Simula 67 (LINDEN, 2008; SEBESTA, 2011).



Ole-Johan Dahl, Kristen Nygaard e Simula

(Regnesentral, 2014)

Outro importante pesquisador que contribuiu para o desenvolvimento da POO foi Alan Curtis Kay, na Xerox, nos EUA. Em seus experimentos, Alan Kay estudava maneiras de interpretar os problemas do mundo real de uma maneira que o ser humano conseguisse abstrair os conceitos fundamentais de tal problema e inferi-los no mundo computacional. Dessa forma, Alan Kay percebeu que um substantivo isolado na mente de uma pessoa gera uma forma concreta, mas um verbo isolado na mente de uma pessoa, não. Logo, um verbo faz parte de um substantivo.

Partindo dessa premissa, torna-se fácil para nós, seres humanos, perceber que "correr", isoladamente, não gera uma informação real (torna-se uma ação sem sentido). No entanto, "crianças correm" gera uma informação real em nossa mente, na qual o substantivo é "crianças" e o verbo é "correr".

A partir dessas observações, Alan Kay definiu os princípios da POO:

- • qualquer coisa no mundo real é um objeto;
- • objetos realizam tarefas por meio de ações;
- • cada objeto pode ser agrupado em tipos (classes);

- um tipo de objeto (classe) deve agrupar objetos por similaridade de forma e comportamento;
- cada tipo de objeto (classe) é organizado hierarquicamente.

As técnicas de POO são cognitivamente validas, visto que a utilização dessas técnicas segue a maneira natural que o ser humano abstrai (imagina) o mundo real. Portanto, a POO é uma metodologia de desenvolvimento de *software* em que a principal vantagem é a proximidade com a forma que os seres humanos visualizam e entendem o mundo ao seu redor, tendo como principal objetivo facilitar a modelagem e o desenvolvimento de sistemas por meio da interação entre objetos.

Alguns exemplos de linguagens modernas utilizadas por grandes empresas em todo o mundo que adotaram essas idéias: Java, C#, C++, Object Pascal (Delphi), Ruby, Python, Lisp, etc.

No entanto, cabe ressaltar que o conceito de Orientação Objeto depende mais da mentalidade do programador do que da linguagem de programação que está sendo utilizada. Pode-se conseguir programas razoavelmente orientados a objeto em linguagens tipicamente estruturadas, assim como pode-se conseguir programas estruturados em linguagens voltadas para objetos. Tomemos como exemplo a frase:

“O navio atraca no porto e descarrega sua carga”.

se analisássemos esta frase estruturadamente, pensaríamos logo em como o navio atraca no porto e como ele faz para descarregar sua carga, ou seja, pensaríamos na ação que está sendo feita (que na frase é representada pelos verbos) para transformá-la em procedimento. Em orientação objeto, o enfoque com que se encara a frase é diferente: primeiro pensaríamos no objeto navio, no objeto porto e no objeto carga, pensando como eles seriam e procurando definir seu comportamento. Após isto é que pensaremos em como o navio se relaciona com o porto e com a carga, e como o porto se relaciona com carga. De modo grosseiro, podemos dizer que ao analisarmos uma frase pensando estruturadamente, damos ênfase aos verbos, e pensando orientado a objetos, damos ênfase aos substantivos.

M1.2 Vantagens e objetivos da OO

Inicialmente vamos olhar para os precursores da programação orientada a objetos (POO). Atualmente, quando usa um computador, você está tirando proveito de mais de 50 anos de refinamento. Antigamente, a programação era engenhosa: os programadores introduziam os programas diretamente na memória principal do computador, através de bancos de chaves (switches). Os programadores escreviam seus programas em linguagem binária, tal programação era extremamente propensa a erros e, a falta de estrutura, tornou a manutenção do código praticamente impossível. Além disso, o código da linguagem binária não era muito acessível. Quando os computadores tornaram-se mais comuns, linguagens de nível mais alto e procedurais começaram; a primeira foi FORTRAN. Entretanto, linguagens procedurais posteriores como ALGOL, tiveram mais influência sobre a orientação a objetos (OO). As linguagens procedurais permitem ao programador reduzir um programa em procedimentos refinados para processar dados. Esses procedimentos geram a execução de um programa procedural. O programa termina quando acaba de chamar sua lista de procedimentos. Esse paradigma apresentou diversas melhorias em relação à linguagem binária, incluindo a adição de uma estrutura de apoio: o procedimento. As funções

menores não são apenas mais fáceis de entender, mas também são mais fáceis de depurar. Por outro lado, a programação procedural limita a reutilização de código. Isso leva os programadores, com muita frequência, a produzir “código de espaguete” – código cujo caminho de execução se assemelhava a uma tigela de espaguete.

Dessa forma, a natureza voltada aos dados da programação procedural causou alguns problemas próprios. Como os dados e o procedimento são separados, não existe nenhum encapsulamento desses dados, isso exige que cada procedimento saiba como manipulá-los corretamente. Infelizmente, um procedimento com comportamento errôneo poderia introduzir erros se não manipulasse os dados corretamente. Uma mudança na representação dos dados exigiria alterações em cada lugar que os acessassem. Assim, mesmo uma pequena alteração poderia levar a uma cascata de alterações, por todo o programa – em outras palavras, um pesadelo de manutenção. A programação modular, com uma linguagem como MODULA2, tenta melhorar algumas das deficiências encontradas na programação procedural. A programação modular divide os programas em vários componentes ou módulos constituintes. Ao contrário da programação procedural, que separa dados e procedimentos, os módulos combinam os dois. Um módulo consiste em dados e procedimentos para manipular esses dados. Quando outras partes do programa precisam usar um módulo, elas simplesmente exercitam a interface do módulo. Como os módulos ocultam todos os dados internos do restante do programa, é fácil introduzir a idéia de estado: um módulo contém informações de estado que podem mudar a qualquer momento. Mas a programação modular possui as seguintes deficiências:

1. Os módulos não são extensíveis, significando que você não pode fazer alterações incrementais em um módulo sem abrir o código fonte “a força” e fazer as alterações diretamente;
2. Os módulos não podem se basear em um outro, a não ser através de delegação;
3. E embora um módulo possa definir um tipo, um módulo não pode compartilhar o tipo de outro módulo. Finalmente, a programação modular também é um híbrido procedural que ainda divide um programa em vários procedimentos. Porém, em vez de atuar em dados brutos, esses procedimentos manipulam módulos.

A Programação Orientada a Objetos (POO) dá o próximo passo lógico após a programação modular, adicionando herança e polimorfismo ao módulo. A POO estrutura um programa, dividindo-o em vários objetos de alto nível. Cada objeto modela algum aspecto do problema a ser resolvido. Escrever listas sequenciais de chamadas de procedimento para dirigir o fluxo do programa não é mais o foco da programação sob a OO. Em vez disso, os objetos interagem entre si, para orientar o fluxo global do programa. De certa forma, um programa OO se torna uma simulação viva do problema. Os objetos nos obrigam verificar, em nível conceitual, tudo o que eles fazem, ou seja, nos fazem listar todos os seus comportamentos. Ver um objeto a partir do nível conceitual não é definir sua implementação (em termos de programação, implementação é o código). Essa mentalidade nos obriga a pensar em programas em termos naturais e reais. Ou seja, os objetos permitem que você modele programas nos substantivos, verbos e adjetivos do domínio de seu problema. Sendo o domínio, o espaço onde um problema reside, o conjunto de conceitos que representam os aspectos importantes do problema.

Quando recua e pensa nos termos do problema que está resolvendo, você evita se emaranhar nos detalhes da implementação. Provavelmente alguns de seus objetos de alto nível precisarão interagir com o computador. Entretanto, o objeto isolará essas interações do restante do sistema. Vamos pensar em um objeto "carrinho de compras". Ocultação da implementação significa que o caixa não vê dados brutos ao totalizar um pedido. O caixa não sabe procurar, em certas posições de memória, números de item e outra variável para um cupom. Em vez disso, o caixa interage com objetos item. Ele sabe perguntar quando custa o item. Antes de prosseguir, como exercício, tente imaginar os objetos item, carrinho de compras e caixa. A seguir você pode ver uma tabela de uma possível representação simplificada dos objetos.

Objeto	Descrição
Item	Representa um produto, pode fornecer dados sobre ele, como preço, foto, descrições, dimensão, peso, quantidade
Carrinho de compras	Armazena itens. Um item pode ser incluído ou retirado.
Caixa	Contabiliza o valor dos itens do carrinho de compras e efetua a compra

Agora já podemos definir formalmente um objeto: Um objeto é uma construção de software que encapsula estado e comportamento. Os objetos permitem que você modele seu software em termos reais e abstração. Rigorosamente um objeto é uma instância de uma classe. Sobre o que é uma classe veremos mais adiante, mas já podemos dizer que uma instância é o objeto propriamente dito alocado em memória. Assim como o mundo real é constituído de objetos, o software orientado a objetos também o é. Em uma programação OO pura, tudo é um objeto, desde os tipos mais básicos, como inteiros e lógicos, até as instâncias de classes mais complexas; nem todas as linguagens orientadas a objeto chegam a esse ponto. Em algumas (como o C# e Java), primitivas "int" e "float" não são tratadas como objetos, assim como no Flash também há limitações.

Com isso, podemos destacar as principais vantagens da Orientação a objetos :

1. Melhor organização do código do que a empregada nos sistemas não orientados a objetos;
2. Facilita a manutenção e diminui a quantidade de erros de programação (bugs);
3. Maior facilidade para reaproveitamento do código;
4. Maior facilidade para o controle de qualidade e teste de software.

Desde sua criação até os dias de hoje, muitas linguagens foram criadas e gradativamente incorporando os conceitos da POO por completo.

A POO pode ser considerada uma evolução da programação estrutural/modelar, haja vista a popularização dos computadores em diversos ambientes com o passar do tempo, ampliando assim o leque de aplicações. Consequentemente, isso aumentou também a demanda na indústria de *software*.

Diferentemente da programação estrutural/modular, na qual temos uma sequência de instruções com interação com o usuário, chamada a sub-rotinas e tomadas de decisão, a POO vai além desse modelo de programação.

Na programação estrutural/modelar, caso o programador queira reutilizar o código de um programa em outro, é feita literalmente uma cópia desse trecho de código para o novo programa. Não há nada de errado em reutilizar códigos; porém, caso o programador tenha reutilizado seu código em diversos programas e alguma especificação for alterada justamente no trecho de código que ele reutilizou, o programador terá que realizar a atualização em todos os programas nos quais ele faz o reuso de código. Com esse exemplo, fica claro que a manutenção em programas torna-se cada vez mais custoso, pois o programador utilizará muitas horas de trabalho para atualizar as aplicações necessárias.

Com a alta demanda na indústria de *software*, as empresas procuram maneiras de minimizar custos e otimizar a produtividade dos programadores. Nesse aspecto, a POO se apresenta como uma ferramenta que auxilia o objetivo de modelagem de sistemas de maneira natural e alta reusabilidade de código, ou seja, o programador modela a solução e escreve o código apenas uma vez, mas utiliza a solução modelada com diversas outras aplicações.

Outro exemplo de vantagem da POO está justamente em sua modelagem de soluções. Por ser uma modelagem com base em abstração do ambiente real, sua estrutura se apresenta com muita facilidade a outros programadores e, portanto, a quantidade de documentação gerada fora da aplicação tende a diminuir, sendo quase autoexplicativo. Podemos ter um breve comparativo entre as vantagens e desvantagens do uso da POO no quadro a seguir.

Vale ressaltar aqui que a POO não isenta o programador a não gerar documentação. O que ocorre é que a documentação gerada pela equipe de desenvolvimento foca mais na regra de negócio do que na explicação da estrutura do programa.

A princípio, não há muita diferença entre a POO e a programação estrutural/modular, mas analisando seus conceitos fundamentais fica claro o quão poderosa é essa técnica de programação.

Quadro 1 – Vantagens e desvantagens de um sistema OO

Vantagens

- Os sistemas POO geralmente possuem uma distribuição de código um pouco mais lógica, funcional e melhor encapsulada, tornando a manutenção e

Desvantagens

- O aprendizado do paradigma de programação orientada a objetos é bem mais complicado no início do que os sistemas estruturados.

extensão do código mais fácil e com menos riscos de inserção de erros.

- É mais fácil reaproveitar o código.
- É mais fácil gerenciar o desenvolvimento deste tipo de *software* em grandes equipes, partindo de uma especificação UML antes de iniciar o desenvolvimento do *software* em si.

- Para começar a programar, é necessário ter uma base conceitual bem formada, ao contrário da programação procedural tradicional, em que basta decorar meia dúzia de comandos para que seja possível fazer programa simples.
- Em termos de velocidade de processamento, dificilmente uma linguagem orientada a objetos conseguirá ter um desempenho superior a linguagens não orientadas a objetos.

Segundo a Microsoft, a programação orientada a objeto (POO) é apoiada em três pilares: encapsulamento, herança e polimorfismo (MICROSOFT, 2014; SINTES, 2002).

Exercício 1:

As maneiras mais comuns de se definir um modelo são :

- A)
perspectiva de um algoritmo e perspectiva orientada a objetos
- B)
perspectiva de um algoritmo e perspectiva orientada a aspectos
- C)
perspectiva de um objeto e perspectiva de um algoritmo
- D)
português estruturado e perspectiva de um algoritmo
- E)
perspectiva de uma classe e perspectiva de um algoritmo

Exercício 2:

Na perspectiva de um algoritmo, o principal bloco de construção de um software é :

A)

classe

B)

procedimento ou função

C)

fluxograma

D)

algoritmo estruturado

E)

void main()

Exercício 3:

B1-2 1

Entre as principais vantagens de POO, podemos citar :

A)

Permite aos programadores introduzirem programas diretamente na memória do computador

B)

Torna o código da linguagem binárias mais acessível

C)

Torna o projeto mais barato

D)

Facilita o entendimento do projeto

E)

Facilita a manutenção e diminui a quantidade de erros de programação

Exercício 4:

B1-2 2

Sobre POO, é correto afirmar que :

A)

Reduz um programa em procedimentos refinados para processar dados

B)

O programa termina quando acaba de chamar sua lista de procedimentos

C)

Apesar de apresentar funções menores, não torna o programa mais fácil de ser entendido

D)

Como os dados e o procedimento são separados, não existe nenhum encapsulamento

E)

Herança é uma de suas carecterísticas

Exercício 5:

B1-2 3

Sobre a programação modular, é errado afirmar que :

A)

Os módulos não são extensíveis

B)

Um módulo consiste em dados e procedimentos para manipular esses dados

C)

Os módulos não podem se basear em um outro módulo

D)

Os módulos podem se basear em um outro módulo

E)

A programação modular necessita dos conceitos da POO

Exercício 6:

Dadas as afirmações:

I-A POO teve origem na década de 1960, na Noruega, no Centro Norueguês de Computação

II-Java, C#, Clipper são exemplos de Linguagens que adotaram as idéias do POO

III-A linguagem C++ por ser antiga ainda não é orientada a objeto

podemos afirmar que:

A)

Somente a a afirmação I está correta

B)

Somente a a afirmação II está correta

C)

Somente a a afirmação III está correta

D)

Mais de uma afirmação está correta

E)

Nenhuma das afirmações está correta

Exercício 7:

Sobre as deficiências da programação modular temos a seguinte lista:

I Os módulos não são extensíveis, significando que você não pode fazer alterações incrementais em um módulo sem abrir o código fonte "a força" e fazer as alterações diretamente;

II. Os módulos não podem se basear em um outro, a não ser através de delegação;

III E embora um módulo possa definir um tipo, um módulo não pode compartilhar o tipo de outro módulo. Finalmente, a programação modular também é um híbrido procedural que ainda divide um programa em vários procedimentos. Porém, em vez de atuar em dados brutos, esses procedimentos manipulam módulos.

Podemos afirmar que:

A)

Somente o item I apresenta uma deficiência da programação modular

B)

Somente o item II apresenta uma deficiência da programação modular

C)

Somente o item III apresenta uma deficiência da programação modular

D)

Mais de um dos itens apresenta a deficiência da programação modular

E)

Nenhum dos itens são deficiências da programação modular

Exercício 8:

Dadas as afirmações:

I. O aprendizado do paradigma de programação orientada a objetos é bem mais complicado no início do que os sistemas estruturados.

II.O reaproveitamento de código é mais fácil.

III.Em termos de velocidade de processamento, sempre uma linguagem orientada a objetos conseguirá ter um desempenho superior a linguagens não orientadas a objetos

Qual das alternativas abaixo é correta

A)

Somente a afirmação I apresenta a vantagem da POO

B)

Somente a afirmação II apresenta a vantagem da POO

C)

Somente a afirmação III apresenta a vantagem da POO

D)

Mais de uma afirmação mostram a vantagem da POO

E)

Nenhuma das afirmações apresenta vantagem da POO